Assignment – 5

- Aditya Kulshreshtha
- Aakulshr

Project Report

Problem Statement

Can deep learning-based clustering approaches provide more actionable and meaningful customer segments than traditional clustering techniques?

Objective

In this project we aim to enhance personalized marketing strategies by identifying meaningful customer segments using advanced clustering techniques. Specifically, we compare traditional approaches like K-Means with deep learning-based methods such as autoencoder-based feature extraction.

Data Source

Name: E-commerce Behaviour Data from Multi-category Store from Kaggle

Link : eCommerce behavior data from multi category store

(**Due to limitations in computing we take data with the timestamp 2020 April and 5 Million Sample data from the entire dataset)

Limitations: One limitation of this dataset is that it only covers interactions from a single month—April 2020. This temporal scope may not capture seasonal trends, promotional impacts, or long-term customer behaviour, and could introduce sampling bias. Future analyses could benefit from incorporating multiple months of data to ensure greater generalizability and robustness of the segmentation.

Key Features

Property	Description
event_time	Time when event happened at (in UTC).
event_type	Only one kind of event: purchase.
product_id	ID of a product
category_id	Product's category ID
category_code	Product's category taxonomy (code name)

brand	Downcased string of brand name.
price	Float price of a product. Present.
user_id	Permanent user ID.
user_session	Temporary user's session ID. Same for each user's session. Is changed every time user come back to online store from a long pause.

Events can be:

1. view - a user viewed a product
2. cart - a user added a product to shopping cart
3. purchase - a user purchased a product

Exploratory data analysis

- *Loaded the dataset (limited to 5 million rows for efficiency).*
- *Removed unneeded columns like event_time and user_session.*
- *Encoded categorical columns (event_type) using label encoding*
- *Transforming top 50 most frequent categories to one hot encoded column*
- *Performed user-level aggregation to compute meaningful behavioral metrics:*
  - *Activity Frequency*
  - *Unique Products Interacted With*
  - *Unique Brands Interacted With*
  - *Average Spend*
- *Visualized boxplots to inspect distributions and identify outliers in each metric.*

Removing unneeded columns

We drop the columns '*event_time*' and '*user_session*' because they do not contribute meaningful information for clustering in their raw form. The 'event_time' column contains timestamp data, which requires feature engineering (such as extracting time-of-day or session duration) to be useful in behavioral analysis. Including it as-is would introduce noise and distort distance calculations. Similarly, 'user_session' is simply a unique session identifier with no inherent behavioral value; it is useful for grouping events but not for modelling user behavior. Removing these columns helps streamline the dataset and focus on features that more directly represent user interactions and purchasing patterns.

```
df = df.drop(columns = ['event_time', 'user_session'])
df
```

✓ 0.3s

Converting categorical values to numeric labels

```
label_encoder = LabelEncoder()
df['event_type_encoded'] = label_encoder.fit_transform(df['event_type'])
df
```

Using LabelEncoder we convert categorical text values (like 'view', 'cart', 'purchase') into numeric labels (like 0, 1, 2). Fitting the label encoder to the event_type column (which include values like 'view', 'cart', 'purchase') and transforms it into numerical codes.

The transformed values are stored in a new column called 'event_type_encoded'.

'cart' = 0 ,'purchase' = 1 , 'view' = 2

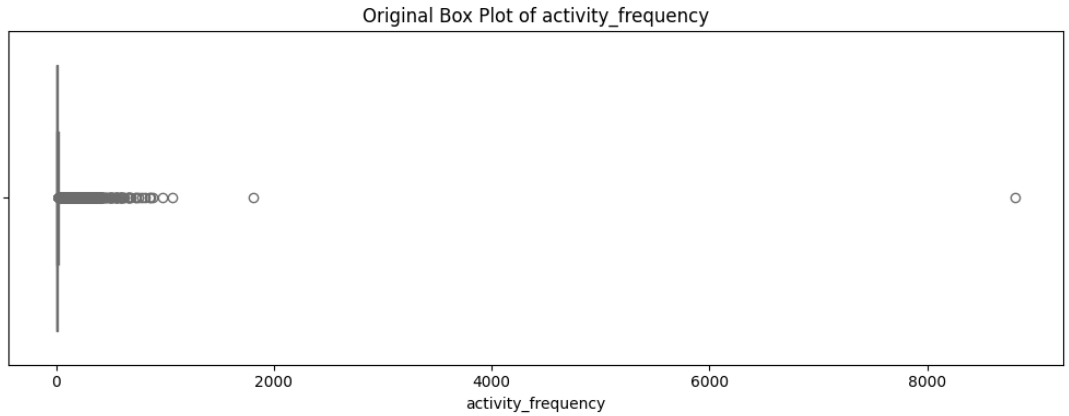
Transforming top 50 most frequent categories to one hot encoded column

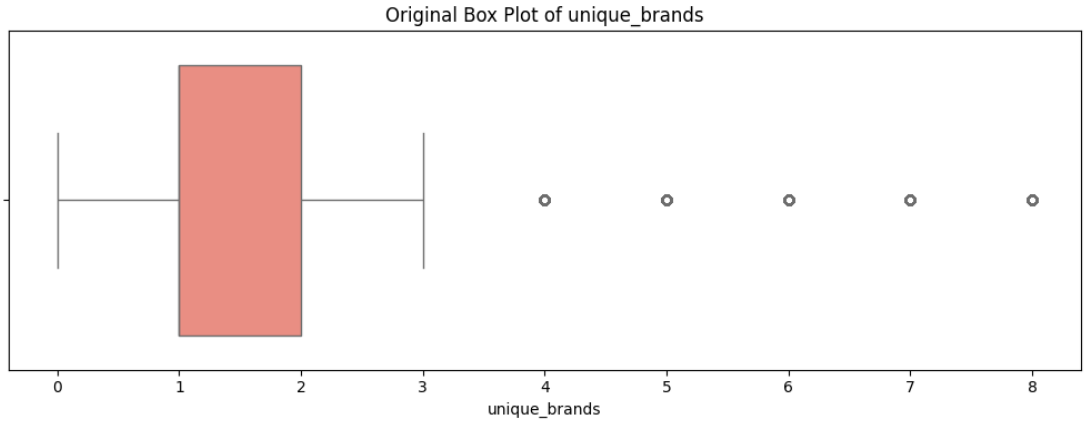
```
top_categories = df['category_code'].value_counts().nlargest(50).index
df['category_code'] = df['category_code'].where(df['category_code'].isin(top_categories), other='other')
one_hot_encoded_df = pd.get_dummies(df['category_code'], prefix='category')
df = pd.concat([df, one_hot_encoded_df], axis=1)
df.drop(columns=['category_code'], inplace=True)

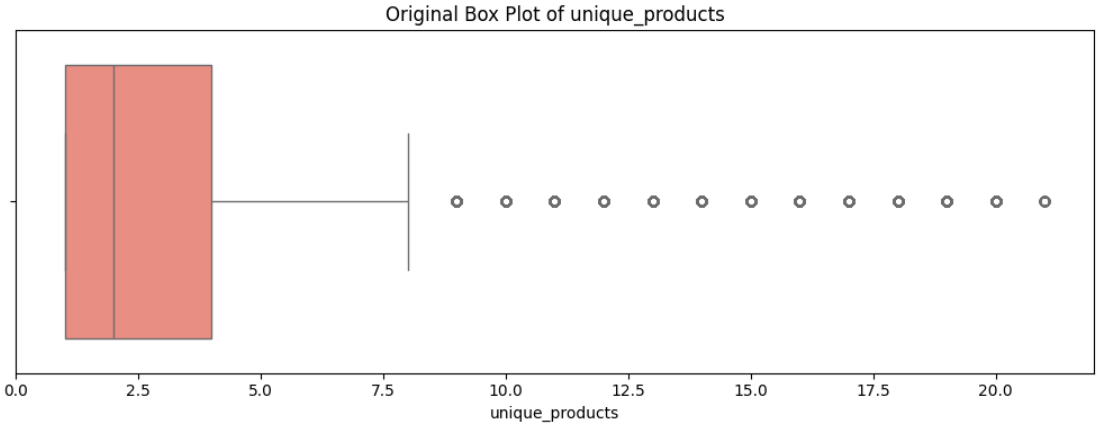
df
```

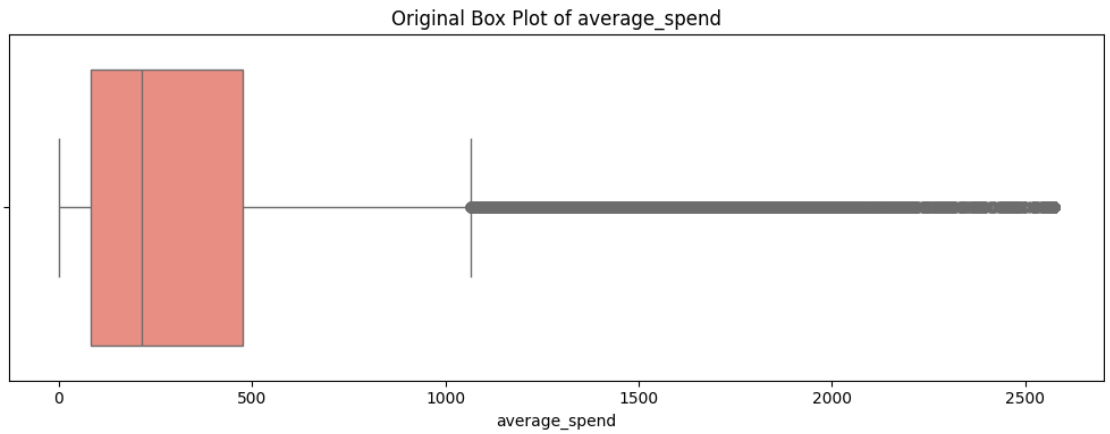
This block of code efficiently manages the high-cardinality category_code feature by retaining only the top 50 most frequent categories and grouping all others under a common label 'other'. It then applies one-hot encoding to the cleaned category_code column, creating binary indicator columns for each retained category (including 'other'). These one-hot encoded features are concatenated back to the original DataFrame to enrich the feature set while preventing the dimensionality from becoming unmanageable. Finally, the original category_code column is dropped, as it is no longer needed in its raw form. This approach ensures that categorical data is encoded in a memory efficient.

Plotting Box-Plots for key features -- pre-Outlier removal:





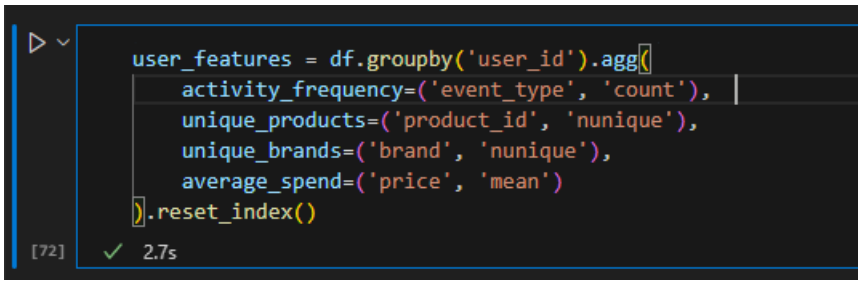




Feature Engineering

- Aggregated transactional data into user-level metrics.
- Applied outlier removal using the Interquartile Range (IQR) method for each feature.
- Standardized the numerical features using StandardScaler to normalize scales for clustering.

Aggregating User behaviour



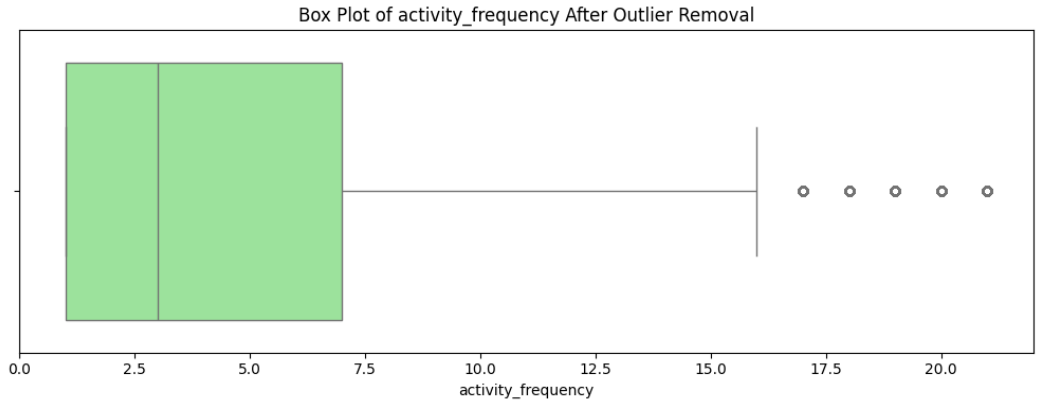
This code block creates a new DataFrame called user_features by aggregating behavioral data at the user level from the original df DataFrame. The aggregation is done using the groupby() function on the user_id column, which allows the computation of summary statistics for each individual user based on their activity history.

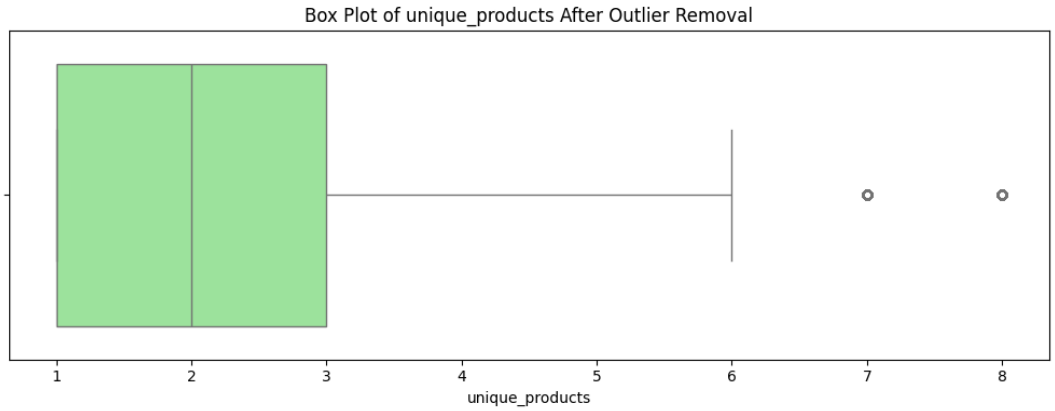
Specifically, the following features are generated:

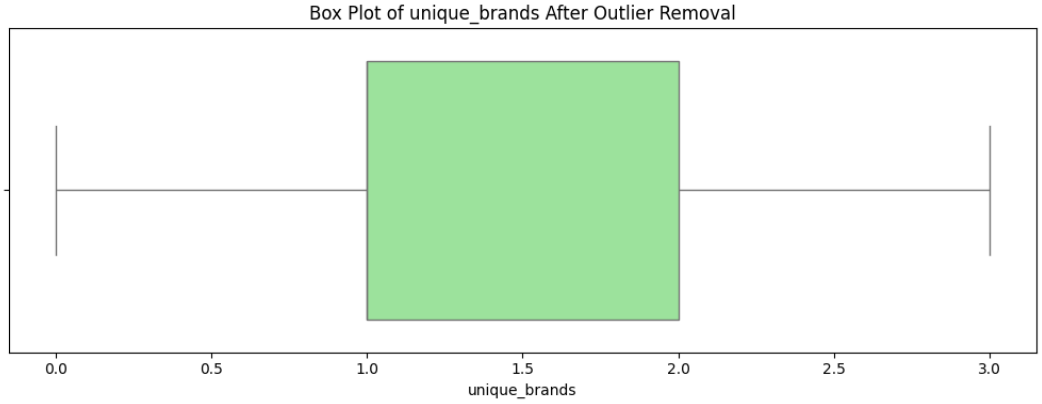
- activity_frequency: This is the total count of events (e.g., view, cart, purchase) associated with a user, calculated using count on the event_type column. It reflects how active or engaged a user is overall.
- unique_products: This captures the number of distinct products a user has interacted with, using nunique on the product_id column. A higher value indicates broader browsing or purchasing behavior.
- unique_brands: This measures how many different brands a user has engaged with, providing insight into brand diversity or loyalty.
- average_spend: This calculates the average price of items the user has interacted with or purchased, serving as a proxy for their typical spending level.

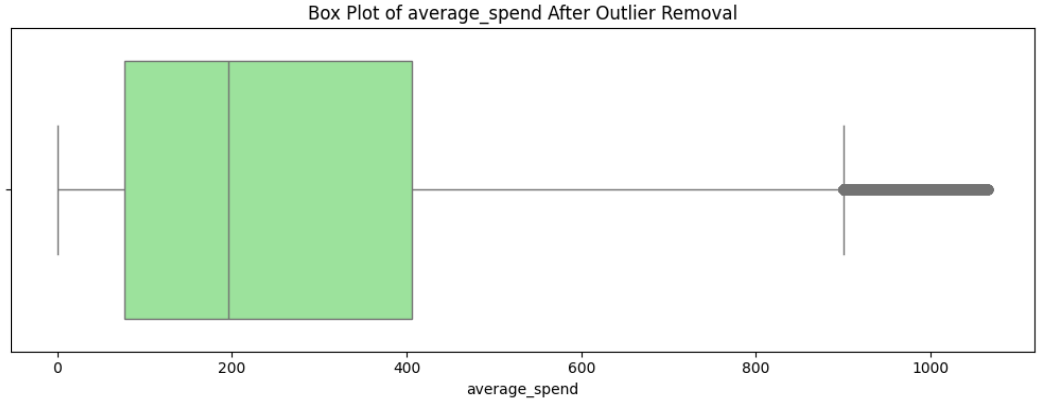
The .reset_index() at the end ensures that user_id remains a column in the resulting DataFrame instead of becoming the index. The final user_features DataFrame represents a condensed summary of each user's behavior, which is ideal for downstream clustering tasks.

Plotting Box-Plots for key features --post-outlier removal:









Outlier removal is performed to improve the quality and reliability of the clustering process by eliminating extreme or anomalous values that could distort the analysis. In clustering algorithms like K-Means, distance metrics (such as Euclidean distance) are used to group data points, and these metrics are highly sensitive to outliers. A few extreme data points can significantly shift cluster centroids, leading to poor cluster formation and misrepresentation of the majority of users.

Standardizing the features

```
features = ['activity_frequency', 'unique_products', 'unique_brands', 'average_spend']

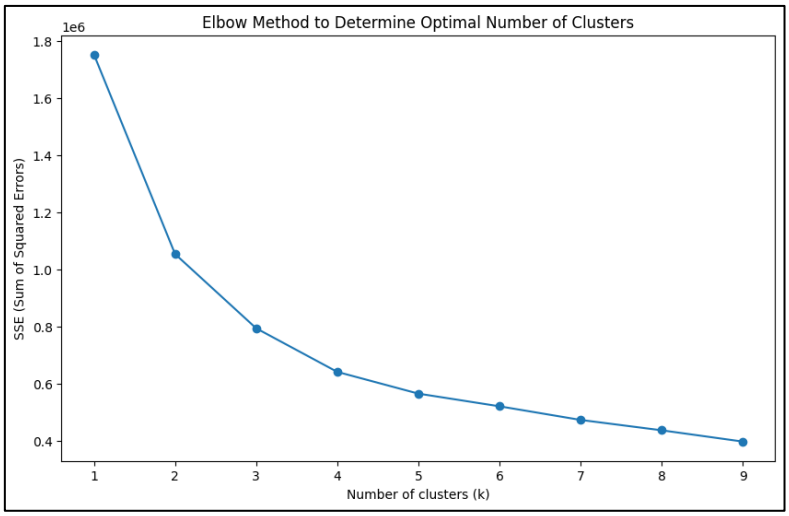
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df_cleaned[features])
```

0.0s

This code block is responsible for preparing user-level behavioral features for clustering by standardizing them. First, it selects a subset of relevant features—activity_frequency, unique_products, unique_brands, and average_spend—which represent different aspects of user engagement and spending behavior. These features are then passed to a StandardScaler, a preprocessing tool from scikit-learn that transforms each feature so that it has a mean of 0 and a standard deviation of 1. This standardization step is crucial because clustering algorithms like K-Means are sensitive to the scale of input features; without it, features with larger numeric ranges could disproportionately influence the clustering outcome. The transformed feature values, now stored in scaled_data, ensure that each dimension contributes equally to the distance calculations used in clustering.

Modeling process

Plotting the Elbow curve to determine the optimal number of clusters for K-Means Clustering



This plot visualizes the results of the Elbow Method, a commonly used technique to determine the optimal number of clusters (k) for K-Means clustering. The x-axis represents the number of clusters, ranging from 1 to 9, while the y-axis shows the Sum of Squared Errors (SSE), which quantifies the total distance between each point and its assigned cluster centroid. As expected, the SSE decreases as the number of clusters increases because the data points are being split into more tightly grouped subsets. However, the key insight lies in identifying the "elbow" point—where the rate of decrease in SSE sharply slows down, indicating diminishing returns from adding more clusters.

For the above specific graph, the elbow appears around k = 4, where the curve starts to flatten noticeably. This suggests that beyond 4 clusters, additional clusters contribute less to improving the compactness of the clustering solution. Therefore, setting k = 4 would likely provide a good balance between simplicity and explanatory power in segmenting the users. The choice between the two may depend on whether the goal is high-level segmentation or more granular behavioural grouping.

This visual validation supported our choice of segmenting users into four distinct behavioral groups, each reflecting unique engagement and spending patterns derived from autoencoder-extracted latent features.

Clustering

```
# Choose the optimal number of clusters, e.g., k = 4 based on the Elbow plot
optimal_k = 4

# Apply K-Means with the optimal number of clusters
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
df_cleaned['cluster'] = kmeans.fit_predict(scaled_data)

# Display the clustered user features
df_cleaned
```

[79] ✓ 0.2s

...

	user_id	activity_frequency	unique_products	unique_brands	average_spend	cluster
0	86517859	1	1	0	36.75	2
1	96847785	1	1	0	548.25	0
2	98232836	1	1	1	909.85	0
3	104655840	1	1	0	9.91	2
5	134184564	1	1	1	142.01	2
...	...	...	...	...	...	...
594946	636498836	2	1	1	687.28	0
594947	636498857	1	1	1	41.06	2
594949	636498871	1	1	1	194.34	2
594950	636498883	2	1	1	687.28	0
594951	636498947	1	1	1	110.62	2

438042 rows × 6 columns

This block of code applies K-Means clustering to the pre-processed user data using the optimal number of clusters determined from the Elbow plot. Based on visual inspection of the curve, the value of k is set to 4, suggesting that four distinct user segments offer a good trade-off between cluster compactness and interpretability. A KMeans object is then initialized with n_clusters=4 and a fixed random_state=42 to ensure reproducibility of results.

The model is fitted on the standardized user feature matrix (scaled_data), and each user is assigned to one of the four clusters based on their behavioral similarity. The resulting cluster labels are stored in a new column named cluster within the df_cleaned DataFrame. This enriched DataFrame now contains both the original features and the corresponding cluster assignments, enabling further analysis of segment characteristics and behavior.

Cluster Size

```
cluster_counts = df_cleaned['cluster'].value_counts()
cluster_counts
```

[80] ✓ 0.0s

```
cluster
2    219977
1     91803
0     74084
3     52178
Name: count, dtype: int64
```

Modeling results

Davies-Bouldin Index

```
from sklearn.metrics import davies_bouldin_score

score = davies_bouldin_score(scaled_data, df_cleaned['cluster'])
print(f"Davies-Bouldin Index: {score:.4f}")
```

[82] ✓ 0.0s

```
Davies-Bouldin Index: 1.0839
```

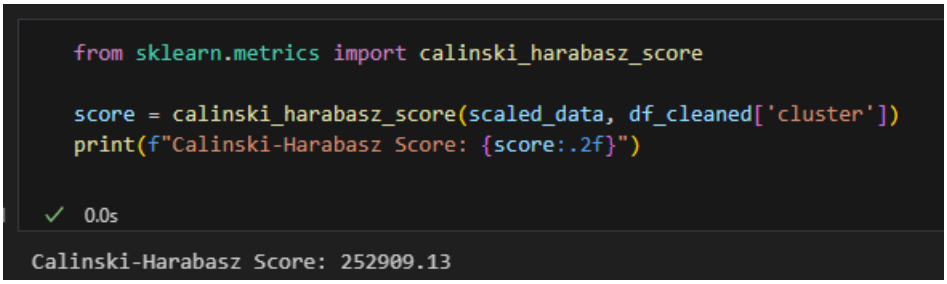
The Davies-Bouldin Index is calculated by:

- Measuring the intra-cluster distance (how close points are within a cluster)
- Comparing that to the inter-cluster distance (how far clusters are from one another)
- Averaging the worst-case ratio for each cluster

Interpretation of Score (1.0839):

- A DBI < 2 is generally considered good, especially on large, real-world datasets.
- Value of ~1.08 indicates that clusters are both reasonably tight internally and fairly well-separated from one another.

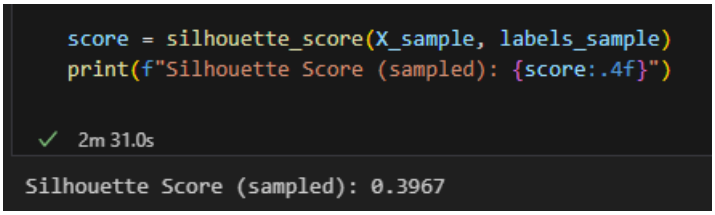
Calinski-Harabasz Index



The Calinski-Harabasz Index evaluates clustering by comparing the between-cluster dispersion to the within-cluster dispersion:

- A higher score means that clusters are dense and well-separated, which is desirable.
- The score tends to scale with the number of samples, so large datasets can naturally produce higher values.
- With a score of 252,909.13, clustering exhibits a very clear separation between clusters, meaning that each group of users is statistically distinct in terms of their behavior.
- This result reinforces that the features used (activity, brand diversity, product diversity, spending) and the latent space from autoencoder are doing a good job of capturing differentiating patterns.

Silhouette score



** For a sample of data

The silhouette score ranges from -1 to 1:

- A score close to 1 indicates that points are well-clustered and far from neighbouring clusters.
- A score close to 0 suggests overlapping clusters or unclear boundaries.

- A negative score would indicate that many points may be assigned to the wrong cluster.

In this case, a silhouette score of 0.3967 suggests moderate cluster structure:

- Clusters are defined, but some overlap or ambiguity exists.
- For customer behavior data, which often has overlapping patterns (e.g., users can show traits of multiple segments), a score in the 0.35–0.45 range is quite common and acceptable.
- This supports the validity of the clustering approach while indicating room for improvement (e.g., better features, more expressive latent space, or fine-tuning k).

Modeling Result – Traditional K-means Clustering

Cluster 0 – Moderate Engagement, Moderate Spend

- Number of users: 108,744
- Average activity frequency: 3.84
- Average unique products: 2.26
- Average spending: $276.93
- Average unique brands: 1.35
- Insight: Users show balanced behavior across spending and engagement but with slightly limited variety.
- Strategy: Encourage brand exploration and increase order size through bundling and product recommendations.

Cluster 1 – Moderate Spenders with Brand Openness

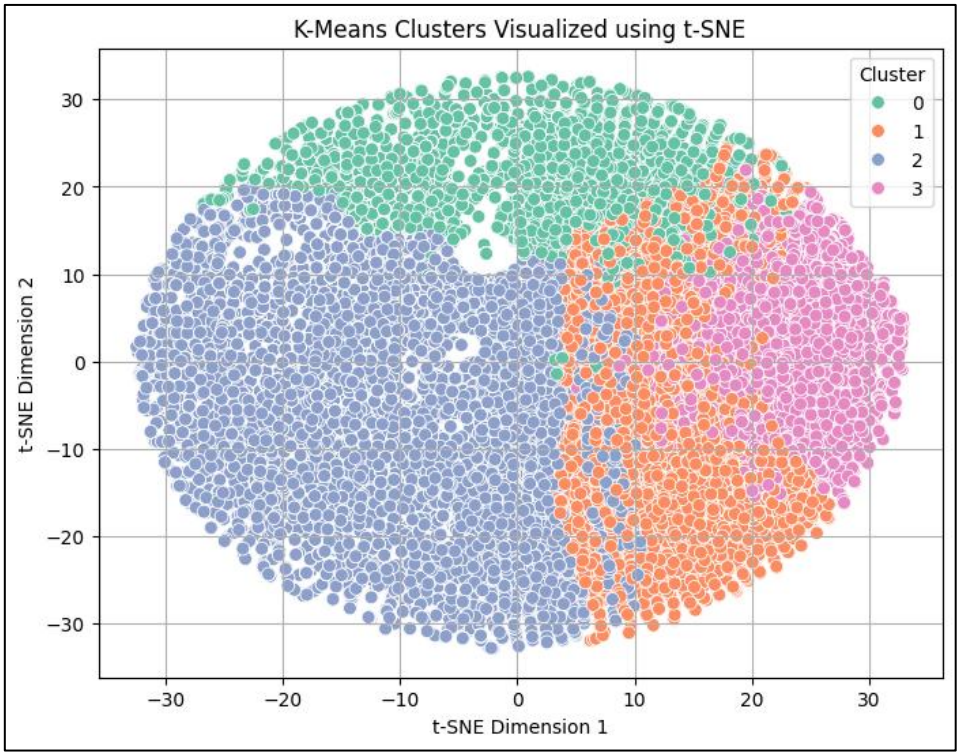
- Number of users: 91,803
- Average activity frequency: 4.42
- Average unique products: 3.01
- Average spending: $205.11
- Average unique brands: 2.19
- Insight: Low-to-mid spenders with moderate engagement and variety in products and brands.
- Strategy: Offer bundle deals and personalized promotions to boost average order value.

Cluster 2 – Large Base of Casual Shoppers

- Number of users: 219,977
- Average activity frequency: 2.16
- Average unique products: 1.27
- Average spending: $155.86
- Average unique brands: 0.86
- Insight: Low-spend, low-engagement users with very limited interactions.
- Strategy: Use welcome campaigns, retargeting, and entry-level offers to increase retention.

Cluster 3 – Engaged and Diverse Shoppers

- Number of users: 52,178
- Average activity frequency: 11.07
- Average unique products: 5.36
- Average spending: $318.26
- Average unique brands: 2.05
- Insight: Active users with above-average purchases and product diversity.
- Strategy: Leverage personalized recommendations, loyalty rewards, and targeted campaigns.



Modelling process – Auto Encoder based K-means Clustering

Model Architecture

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import StandardScaler
import numpy as np

# X = your standardized input features (scaled_data from Phase 1)
input_dim = X.shape[1]

# Define encoder
input_layer = Input(shape=(input_dim,))
encoder = Dense(64, activation='relu')(input_layer)
encoder = Dense(32, activation='relu')(encoder)
latent = Dense(10, activation='relu')(encoder)

# Define decoder
decoder = Dense(32, activation='relu')(latent)
decoder = Dense(64, activation='relu')(decoder)
output_layer = Dense(input_dim, activation='linear')(decoder)

# Compile model
autoencoder = Model(inputs=input_layer, outputs=output_layer)
autoencoder.compile(optimizer=Adam(), loss='mse')

# Train the model
autoencoder.fit(X, X, epochs=50, batch_size=256, validation_split=0.1, verbose=1)
```

Autoencoder Architecture for Feature Extraction

To capture latent patterns in user behavior, we used a deep autoencoder:

- Input: Standardized user features (e.g., activity frequency, product and brand diversity, spending).
- Encoder: Three dense layers (64 → 32 → 10 neurons, ReLU) compress data into a 10-dimensional latent space.
- Decoder: Mirrors the encoder to reconstruct input using ReLU and linear activation.
- Training: 50 epochs, batch size 256, 10% validation split, optimized with Adam and MSE loss.

After training, we used the encoder to extract latent features, which were then clustered using K-Means for behavior-based segmentation.

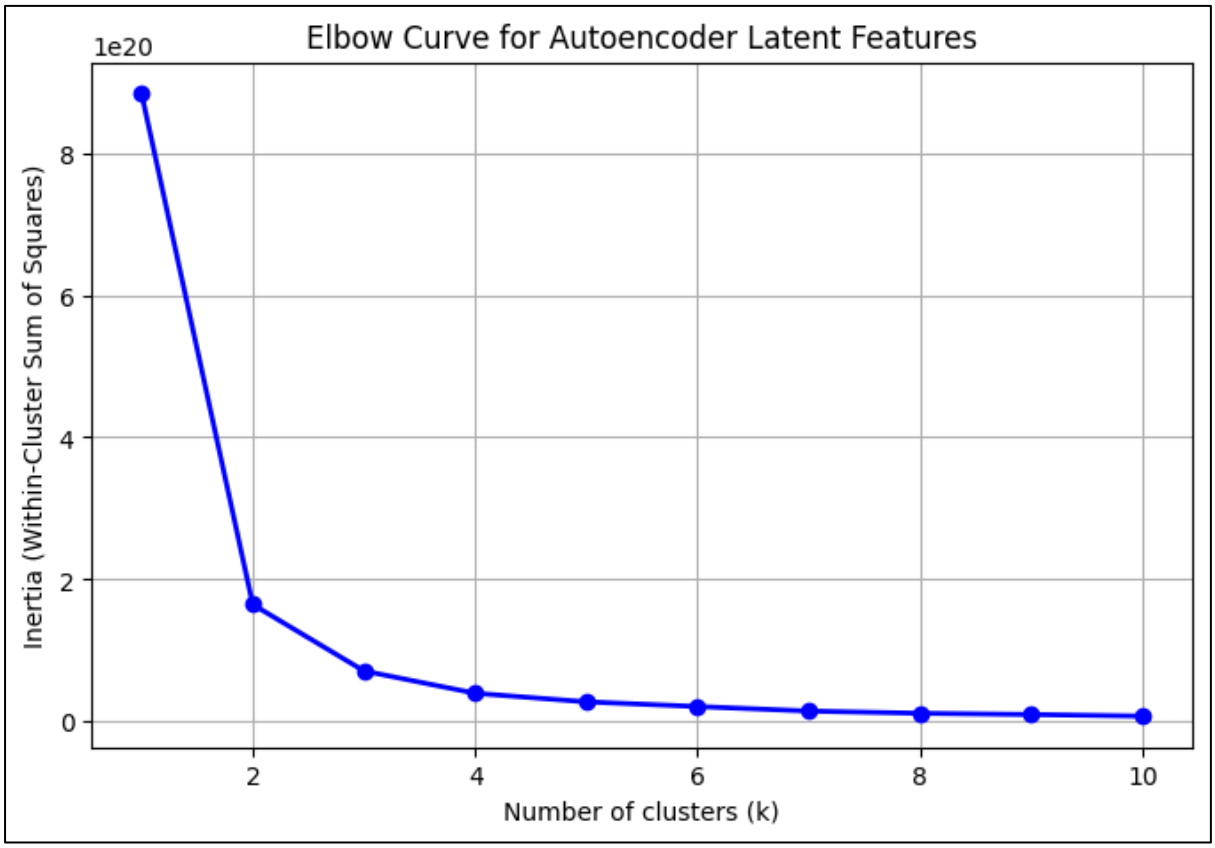
Extracting Latent feature/Feature Engineering

```
encoder_model = Model(inputs=input_layer, outputs=latent)
X_latent = encoder_model.predict(X)
```

- This creates a new model that maps input features to the 10-dimensional latent space.
- X_latent is the compressed representation of user behavior, which captures the essential patterns for clustering.

These latent vectors are then used as input for K-Means clustering, enabling more accurate and expressive customer segmentation.

Plotting the Elbow curve to determine the optimal number of clusters for Autoencoder based K-means clustering



Elbow Curve for Autoencoder Latent Features

The elbow curve above visualizes the within-cluster sum of squares (inertia) for different numbers of clusters (*k*) applied to the autoencoder-extracted latent features.

- A steep drop in inertia occurs between *k = 1* and *k = 2 continuing till k= 3*, indicating that clustering meaningfully reduces variance.
- The curve begins to flatten from *k = 3* onwards, forming an "elbow" — the point beyond which additional clusters yield minimal improvement.

Applying K-means on Latent features

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(X_latent)
```

Modelling Result – Auto Encoder Clustering

```
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score

print("Silhouette Score:", silhouette_score(X_latent, clusters))
print("Calinski-Harabasz Index:", calinski_harabasz_score(X_latent, clusters))
print("Davies-Bouldin Index:", davies_bouldin_score(X_latent, clusters))
```

```
Silhouette Score: 0.67404205
Calinski-Harabasz Index: 2543269.8
Davies-Bouldin Index: 0.4493720835274349
```

To evaluate the effectiveness of the K-Means clustering applied to the latent features extracted via the autoencoder, we employed three standard clustering performance metrics:

- Silhouette Score: 0.674
  The silhouette score assesses how similar each data point is to its assigned cluster relative to other clusters. It ranges from -1 (poor fit) to 1 (ideal clustering).
    - *Interpretation:* A score of 0.674 indicates well-defined clusters with strong internal cohesion and good separation from other clusters.
- Calinski-Harabasz Index: 2,543,269.8
  This metric measures the ratio of between-cluster dispersion to within-cluster dispersion. Higher values denote better clustering quality.

- *Interpretation:* The very high value of over 2.5 million suggests that the clusters are compact internally and well-separated from each other.

- Davies-Bouldin Index: 0.449
  This index evaluates the average similarity between each cluster and its most similar one. Lower values represent better clustering.
  - *Interpretation:* A score below 0.5 reflects excellent cluster distinctiveness and internal consistency.

Cluster 0 – Moderate Engagement, Moderate Spend

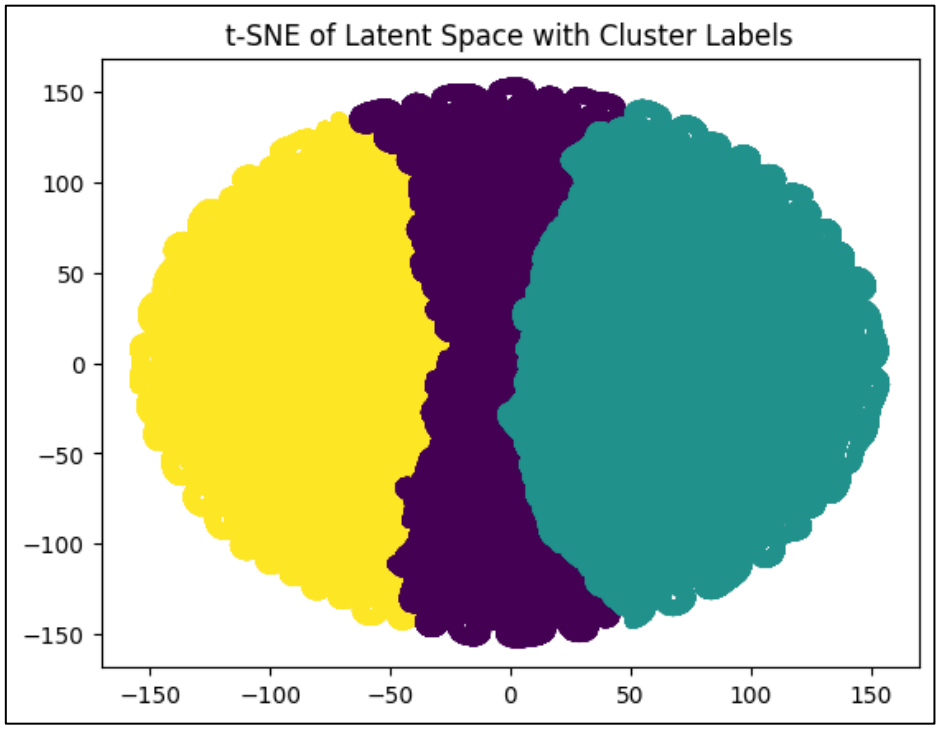
- Number of users: 108,744
- Average activity frequency: 3.84
- Average unique products: 2.26
- Average spending: $276.93
- Average unique brands: 1.35
- Insight: Users show balanced behavior across spending and engagement but with slightly limited variety.
- Strategy: Encourage brand exploration and increase order size through bundling and product recommendations.

Cluster 1 – Large Base, Conservative Behavior

- Number of users: 189,162
- Average activity frequency: 3.48
- Average unique products: 2.02
- Average spending: $278.86
- Average unique brands: 1.26
- Insight: Consistent but conservative buyers with average interest in brand diversity.
- Strategy: Use loyalty incentives, suggest complementary products, and reinforce brand trust.

Cluster 2 – Low-Value, Infrequent Shoppers

- Number of users: 140,136 (largest group)
- Average activity frequency: 4.07
- Average unique products: 2.37
- Average spending: $282
- Average unique brands: 1.39
- Insight: Slightly more active users exploring a broader range of products and brands.
- Strategy: Encourage engagement with seasonal promotions, new arrivals, and curated brand experiences.



Main Insight

To evaluate the effectiveness of deep learning-enhanced clustering, we compared two distinct approaches:

- Traditional K-Means Clustering on standardized behavioral features (k = 4)
- Autoencoder-Based Clustering on compressed latent features (k = 3)

Cluster Quality Metrics Comparison

Metric	**Traditional Clustering**	**Auto-Encoder based Clustering**
Davies-Bouldin Index	1.0839	0.4493
Calinski-Harabasz Score	252,909	2,543,269
Silhouette Score	0.3967	0.6740

Observation

The autoencoder-based clustering significantly outperforms traditional clustering across all quality metrics, demonstrating better separation between clusters and higher internal cohesion.

Davies-Bouldin Index

- Lower is better.
- Measures how similar clusters are to each other.
- Autoencoder model scores 0.45, a major drop from 1.08, meaning the clusters are much tighter and well-separated.

Calinski-Harabasz Score Index

- Higher is better.
- Measures the ratio of between-cluster variance to within-cluster variance.
- The autoencoder gives a massive boost from 252,909 to 2.5 million+, showing significantly better-defined clusters.

Silhouette Score

- Ranges from -1 to +1.
- Higher scores mean that the points are well matched to their own cluster and far from others.
- Improved from 0.3967 to 0.6740, again confirming clearer clustering boundaries.

Visual Comparison

Elbow Method

- Traditional Clustering: The elbow at $k = 4$ was modest, suggesting incremental improvement.
- Autoencoder Clustering: A sharp drop in inertia followed by a plateau at $k = 3$ indicates more distinct segmentation.

t-SNE Visualization

- Traditional Clustering (t-SNE): Clusters show overlap and indistinct boundaries.
- Autoencoder Clustering (t-SNE): Clusters are compact, symmetric, and well-separated.

These visualizations support the metric-based conclusions, emphasizing the benefit of using learned feature representations for segmentation.

Conclusion

Autoencoder-based clustering provides substantial improvements in:

- Clustering precision (tighter, more consistent groups)
- Interpretability (clearer segment separation)
- Business actionability (more distinct customer profiles)

By compressing user behavior into a low-dimensional latent space, this approach uncovers deeper, non-linear patterns that traditional methods fail to detect.

As a result, autoencoder-based clustering is highly effective for high-dimensional customer analytics, especially when segmentation quality significantly influences personalization, targeting, and revenue outcomes. While this method excels in producing tighter, well-separated clusters, it may abstract some of the interpretable, business-relevant differences among groups.

In contrast, traditional clustering methods, though slightly less precise, often yield segments with more intuitively understandable KPI differences, which can facilitate quicker strategy development and stakeholder communication.

In conclusion, the choice between traditional and autoencoder-based clustering should align with the business objective:

- Use autoencoder-based clustering when precision and behavioral nuance are key.
- Prefer traditional clustering when interpretability and strategy derivation are prioritized.

Both methods are valuable and complementary, depending on the level of insight and granularity needed for downstream applications.